1 2 9 0

FACULDADE DE
CIÊNCIAS E TECNOLOGIA
UNIVERSIDADE D
COIMBRA

Departamento de Engenharia Informática

Intelligent Autonomous Agents
2025/2026

2nd Project
Pac-Man's getting old

Fraud denotes a serious lack of ethics and represents unacceptable behavior for a higher education student. Any attempt at fraud will result in the cancellation of the practical component for both the facilitator and the perpetrator, regardless of any additional disciplinary actions that may be taken under current legislation. If non-original material is used, the **sources** must be explicitly cited. If you utilize AI tools in the production of this work (e.g., ChatGPT), you must clearly specify all aspects in which the tool was involved. Note that, during the defense, you must demonstrate a deep understanding of the content generated by the tool, and this knowledge will be subject to evaluation.

1 Introduction

The second project builds on top of the previous one, where our goal is still to implement an agent for controlling Pac-Man. Many years have passed since you implemented the reactive Pac-Man agent. Currently, Pac-Man is getting old and his eyesight is not what it used to be. It can still see the pellets just fine, but struggles to see the ghosts. Fortunately, his memory and sense of hearing are quite good, enabling it to estimate the position of ghosts far away.

2 Tasks

The goal of this assignment is for you to help Pac-Man solve the maze and eat all the pellets, while avoiding a ghost. To do so, you need to take advantage of the knowledge acquired during the Intelligent Autonomous Agents class and implement a Hidden Markov Model (HMM) to estimate the position of the ghost. You can then adapt your previous reactive agent, or implement a completely new one, using the belief state of the HMM as the base to build your agent's ghost perceptions.

To do so, you must use the TP2 setup in pacman.py, as per the listing below.

```
map_file='maps/originalClassic-single-ghost.txt'
ghost_true_prob = 0.5 # probability of the ghost sensor yielding an accurate reading
n_experiments = 10

score = np.zeros(n_experiments)
for i in range(n_experiments):
    pacman_policy = agents.pacmanHMM
    ghost_policies = [agents.blinky_agent, agents.pinky_agent, agents.inky_agent, agents.clyde_agent]
    frightened_ghost_policies = [agents.random_walk for _ in range(4)]
    score[i] = game_engine.main(pacman_policy, ghost_policies, frightened_ghost_policies, map_file=map_file, ghost_true_prob=ghost_true_prob)
print(f"Average score of {n_experiments} experiments: ", np.mean(score))
```

In order to make things simpler, track the position of a single ghost, using a modified version of the map from the previous work (originalClassic-single-ghost.txt). As in the first project, your agent is to be implemented in the agents.py function, as per the listing below.

```python
def calculate_observation_probability(valid_positions,s, o, true_prob):
    #TODO: Implement the sensor model P(O_t | X_t) for the ghost
    #s = ghost state, o = observation
    #true_prob is the probability of the sensor yielding an accurate reading
    #should return a single probability value of the sensor reading given the ghost's position
    pass

def bayesian_filter(observation, model):
    #TODO: Implement the Bayesian filter to update the belief vector.

    #1. PREDICTION: Compute the belief using transition model

    #2. UPDATE: Incorporate evidence using observation model

    #3. NORMALIZATION: Ensure the belief map is a probability distribution
    pass

def pacmanHMM(game_state):
    if game_state['pacman']['model'] is None:
        model = {}
        model['num_observations'] = model['num_states'] = len(game_state['valid_positions'])

        #TODO: complete the code below to initialise the model. The belief vector is already provided with a uniform prior.

        #define the belief vector. To simplify, we will use a 1D vector of size num_states
        #(i.e., the flattened grid of cells with no obstacles)
        model['belief'] = np.full(model['num_states'], 1.0/model['num_states']) #1D initial state distribution.

        #1. Define the Transition Matrix (T) - models the probability of moving from one state to any other state

        #2. Define the observation Matrix (E) - models the likelihood of sensor readings given the ghost's position

        ##visualisation
        if game_engine.VISUALISE:
            plt.ion()
```

```
211                 fig, ax = plt.subplots()
212                 cax = ax.imshow(vector_to_matrix(model['belief'],
                        game_state['grid_size'], game_state['
                        valid_positions']), cmap='hot', interpolation='
                        nearest')
213                 plt.colorbar(cax, label='Probability')
214                 ax.set_title('Ghost-Belief')
215                 model['fig'] = fig
216                 model['ax'] = ax
217                 model['cax'] = cax
218
219             game_state['pacman']['model'] = model
220
221         #get the noisy ghost observation
222         observation = game_state['valid_positions'].index(
                pacman_perceptions.noisy_ghost_position_sensor(game_state
                , 0, game_state['pacman']['ghost_true_prob']))
223
224         #apply the bayesian filter to update the belief
225         bayesian_filter(observation, game_state['pacman']['model'])
226
227         if game_engine.VISUALISE:
228             visualise_belief(vector_to_matrix(game_state['pacman']['
                    model']['belief'], game_state['grid_size'],
                    game_state['valid_positions']), pacman['model']['cax'
                    ])
229
230         #TODO: Use the updated belief in a reactive agent to move
                Pac-Man
231         #replace this line by your own implementation
232         random_walk(game_state['pacman'], game_state)
```

As you can see, there are three functions to complete. The main one is pacmanHMM, which should start by estimating the position of the ghosts and later use that information along with the remaining observations to select an action. To do so, you must first complete the code for creating the model (lines 194 to 219). The prior belief state is already provided as a 1D vector of states (the grid of valid locations is flattened to create this vector). You must implement the transition model as well as the sensor model. To do so, you will also need to implement the calculate_observation_probability function in line 176, which should return the probability of the sensor yielding a specific observation o given a state s.

Assuming that the model is created, the noisy_ghost_position_sensor (line 222) function is called to obtain a noisy reading of the ghost position. The 'ghost_true_prob' field of pacman's object is used as the probability of this sensor yielding the accurate ghost position, returning a random location

otherwise.

After obtaining the observation, the bayesian_filter function is called (line 225) for estimating the current ghost position based on the observation obtained. As the name of the function suggests, the estimation of the ghost position is made through the process of bayesian filtering, which you should implement in the function from line 183.

It is important to note that Pac-Man's perception of the ghost's position is noisy. Pac-Man can hear the ghost anywhere in the environment, and he can guess its position, but its guesses are far from perfect. In fact, Pac-Man can only acurately detect the ghost's position 50% of the time, with the remaining 50% being evenly distributed between the remaining cells of the environment that do not contain walls. This noisy sensor is implemented in line 106 of pacman_perceptions.py, as shown in the listing below.

```
106 def noisy_ghost_position_sensor(game_state, ind_ghost, prob):
107     #the sensor returns the accurate position of the ghost with
            probability prob
108     #and a random valid position otherwise (excludes cells with
            walls and the ghost's current position)
109     if random.random()<prob:
110         return (game_state['ghosts'][ind_ghost]['x'], game_state
                ['ghosts'][ind_ghost]['y'])
111     else:
112         candidates = game_state['valid_positions'][:]
113         candidates.remove((game_state['ghosts'][ind_ghost]['x'],
                game_state['ghosts'][ind_ghost]['y']))
114         return random.choice(candidates)
```

For the remaining perceptions (e.g., walls and pellets), you may use the functions defined in pacman_perceptions.py. Note that, as in the previous work, **performance matters. That is not to say that the grade is proportional to the score and success rates obtained, but rather that you should put some effort into designing a reactive agent that performs as well as possible. Do not forget to document your process in the report, including the results, reasons behind the shortcomings and approaches for overcomming them.**

After having implemented your HMM-based agents, inspect the belief state over the course of the simulation. How do different sensor probabilities affect them? To answer this question you must **test each of your agents with various sensor probabilities, as follows 10%, 25%, 50% 75% and 90%. For each sensor probability, run a minimum of 10 experiments and analyse the average score as well as the success rate (i.e., the percentage of games won). Also, perform a qualitative analysis of the belief states and the behaviours exhibited by the**

agents at different sensor probabilities.

As an **extra challenge**, devise an HMM to also model the ghost state (i.e., frightened or not) and include this knowledge in your best agent. Describe the changes made to it and test it for the previously described levels of noise. Note that this challenge is not directly marked (i.e., you may attain a 100% mark without doing it). However, it will be used to regain points from possible shortcomings (up to a maximum of 10% of the grade). **If you opt to do this challenge, the noisy_sensor function implemented in line 99 of the pacman_perceptions.py script may be useful.**

3 Submission

This assignment is to be developed by students individually. It is due on the 13th of April 2026 and the submission shall be done in InforEstudante. No delayed submissions will be accepted, nor will any other method of submitting.

Regarding the materials to submit, you must deliver the **modified code, with your implementation of the reactive agent, as well as a brief report (10 pages max.) containing a detailed description of your implementations (including the HMM, the perceptions built upon it and the reactive agents) as well as the analysis of the results and development process. If you attempt the extra challenge, you should also describe your implementation of the HMM for the ghost state as well as the changes made to the reactive agent and its performance analysis.**

4 References

1. **Inteligência Artificial: Fundamentos e Aplicações**
   Ernesto Costa, Anabela Simões

2. **Artificial Intelligence: A Modern Approach**
   Stuart Russel, Peter Norvig